CSE 482 Project

Predicting NASCAR Driver Performance Using Historical Race Statistics

Team Members: Arjun Ray, Aaryaman Bisht, Adhiraj Nimbalkar, Tolu Fashina, Noah Zemba

Collection and storing the data

Target Domain:

Our project focuses on NASCAR race analytics, specifically predicting race performance outcomes using historical driver and race data.

- Given a set of drivers participating in the same race, predict which driver will finish ahead, and approximate positions for each driver
- Given a set of drivers starting from different starting positions, will the starting affect race-end psoitions.
- Does recent form matter more than long term average?
- Predict positions based on track type, location and difficulty.
- Does car driven affect driver position?

We used publicly available NASCAR data from SportsDataIO. The SportsDataIO NASCAR data dictionary was used to understand schema structure, variable meanings, and relationships between tables.

In [8]:
```
## Data Preprocessing
## Describe the original data quality after loading dataset -- details of th
## Clean and improve data, what strategies you've tried to improve it, and a
## Explain techniques used to clean and improve
```

In [9]:
```
import os
import json
import time
import requests
import pandas as pd
```

In [10]:
```
apikey="960411d9b7bf464794671691c80893fb"
if not apikey:
    raise ValueError("Missing API key. Please set SPORTSDATAIO_KEY in your e
BASE="https://api.sportsdata.io/nascar/v2/json"
HEADERS={"Ocp-Apim-Subscription-Key": apikey}
SEASONS=[2025]
SLEEP_SEC=0.25
```

```
os.makedirs("data/raw",exist_ok=True)
os.makedirs("data/processed")
```

In [11]:
```
import pandas as pd
df=pd.read_csv("sportsdataio-nascar-data-dictionary.csv")
df.head()
```

Out[11]:

	TableID	TableName	Name	DataType	Size	Nullable	Description	IsInCSV
0	8001000	Series	SeriesID	integer	32.0	False	The unique ID of this series	True
1	8001000	Series	Name	string	50.0	False	The name of this series	True
2	8002000	Race	RaceID	integer	32.0	False	The unique ID of this race	True
3	8002000	Race	SeriesID	integer	32.0	False	The unique ID of the series that this race is ...	True
4	8002000	Race	SeriesName	string	50.0	False	The name of the series that this race is assoc...	True

In [12]:
```
def get_json(url):
    r=requests.get(url, headers=HEADERS, timeout=60)
    if r.status_code!=200:
        raise RuntimeError(f"Request failed ({r.status_code}).\nURL: {url}\r
    return r.json()

def save_raw(obj, filename):
    path=os.path.join("data/raw", filename)
    with open(path,"w",encoding="utf-8") as f:
        json.dump(obj,f,indent=2)
    return path
```

In [13]:
```
all_races = []
for season in SEASONS:
    url=f"{BASE}/races/{season}"
    print("Fetching races for", season)
    races=get_json(url)
    save_raw(races, f"races_{season}.json")
    all_races+=races
    time.sleep(SLEEP_SEC)

races_df=pd.DataFrame(all_races)
print("Raw races rows:", len(races_df), "cols:", len(races_df.columns))
```

```
wc=["RaceID", "SeriesID", "SeriesName", "Season", "Name", "Day", "DateTime",
keep_cols=[c for c in wc if c in races_df.columns]
races_df=races_df[keep_cols].copy()
if "DateTime" in races_df.columns:
    races_df["DateTime"] = pd.to_datetime(races_df["DateTime"], errors="coer

if "IsOver" in races_df.columns:
    races_df=races_df[races_df["IsOver"]==True].copy()
races_df=races_df.dropna(subset=["RaceID"]).drop_duplicates(subset=["RaceID"
out_path = "data/processed/races.csv"
races_df.to_csv(out_path, index=False)
print("Saved:",out_path,"rows=",len(races_df))
```

```
Fetching races for 2025
Raw races rows: 99 cols: 20
Saved: data/processed/races.csv rows= 99
```

In [14]:

```
drivers_url = f"{BASE}/Drivers"
print("Fetching drivers...")
drivers = get_json(drivers_url)
save_raw(drivers, "drivers.json")
drivers_df = pd.DataFrame(drivers)
wc=["DriverID", "FirstName", "LastName", "Number", "Team", "Manufacturer"]
keep=[c for c in wc if c in drivers_df.columns]
drivers_df = drivers_df[keep].drop_duplicates(subset=["DriverID"])
out_path = "data/processed/drivers.csv"
drivers_df.to_csv(out_path, index=False)
print("Saved:", out_path, "rows=", len(drivers_df))
```

```
Fetching drivers...
Saved: data/processed/drivers.csv rows= 843
```

In [15]:

```
races_final = pd.read_csv("data/processed/races.csv")
race_ids = races_final["RaceID"].dropna().astype(int).tolist()
all_rows = []
failed = []
for rid in race_ids:
    url=f"{BASE}/raceresultfinal/{rid}"
    try:
        rr = get_json(url)
        save_raw(rr, f"raceresultfinal_{rid}.json")
        race_obj = rr.get("Race", {}) if isinstance(rr, dict) else {}
        driver_races = rr.get("DriverRaces", []) if isinstance(rr, dict) els
        for dr in driver_races:
            all_rows.append({
                "RaceID": race_obj.get("RaceID", rid),
                "Season": race_obj.get("Season"),
                "SeriesName": race_obj.get("SeriesName"),
                "DateTime": race_obj.get("DateTime"),
                "Track": race_obj.get("Track"),
                "DriverID": dr.get("DriverID"),
                "StartPosition": dr.get("StartPosition") or dr.get("Starting
                "FinishPosition": dr.get("FinishPosition") or dr.get("FinalP
                "Laps": dr.get("Laps") or dr.get("LapsCompleted"),
                "Points": dr.get("Points"),
            })
```

```
        time.sleep(SLEEP_SEC)

    except Exception as e:
        failed.append((rid, str(e)))

results_df = pd.DataFrame(all_rows)
print("Raw results rows:", len(results_df))
results_df["DateTime"] = pd.to_datetime(results_df["DateTime"], errors="coer
num_cols = ["RaceID", "DriverID", "StartPosition", "FinishPosition", "Laps",
for col in num_cols:
    if col in results_df.columns:
        results_df[col] = pd.to_numeric(results_df[col], errors="coerce")
results_df = results_df.dropna(subset=["RaceID", "DriverID", "FinishPositior
results_df = results_df.drop_duplicates(subset=["RaceID", "DriverID"])
results_df["Top10"] = (results_df["FinishPosition"] <= 10).astype(int)

out_path = "data/processed/race_results.csv"
results_df.to_csv(out_path, index=False)

print("Saved:", out_path, "rows=", len(results_df))
print("Failed races:", len(failed))
if failed:
    print("First failure:", failed[0])
```

```
Raw results rows: 3667
Saved: data/processed/race_results.csv rows= 3584
Failed races: 0
```

We retrieved the data using the API and stored it in pandas DataFrames. From the retrieved data, a clean and structured data set was formed by combining the necessary data. This was done by matching the race columns and ensuring the data sets were compatible.

The original data set had missing information and a few columns that were not necessary for the model's predictions. The unwanted attributes were removed from the data set. The columns were also made uniform in name and data type for a smooth transition in the next phase.

New columns were added to the data set for easier analysis. For instance, new columns were added for easier classification. The data was also reorganized so that each row corresponds to a driver and a race.

This resulted in a clean and tabular data set ready for machine learning. The clean data set was ready for the next phase in the process.